

ECE433 – Γραφικά ΗΥ

Απόδοση Φωτισμού & Υλικού

Κ. ΝΤΑΛΟΥΚΑΣ – Π. ΤΣΟΜΠΑΝΟΠΟΥΛΟΥ

ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΙΑΣ

ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧ & ΜΗΧ. ΥΠΟΛΟΓΙΣΤΩΝ

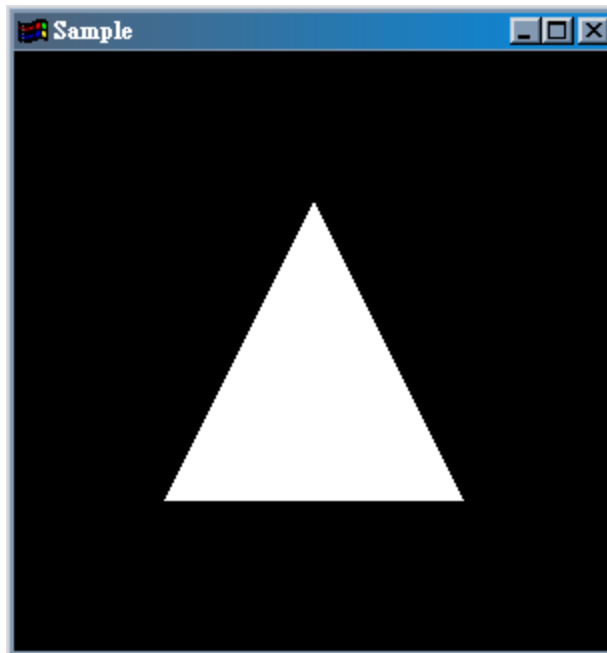
Color

- ▶ Two Color modes:
 - ▶ RGBA
 - ▶ Color Index
- ▶ Choosing between RGBA and Color Index
 - ▶ **glutInitDisplayMode(); (GLUT)**
 - ☐ **glutInitDisplayMode(GLUT_RGBA)**
 - ☐ **glutInitDisplayMode(GLUT_INDEX)**

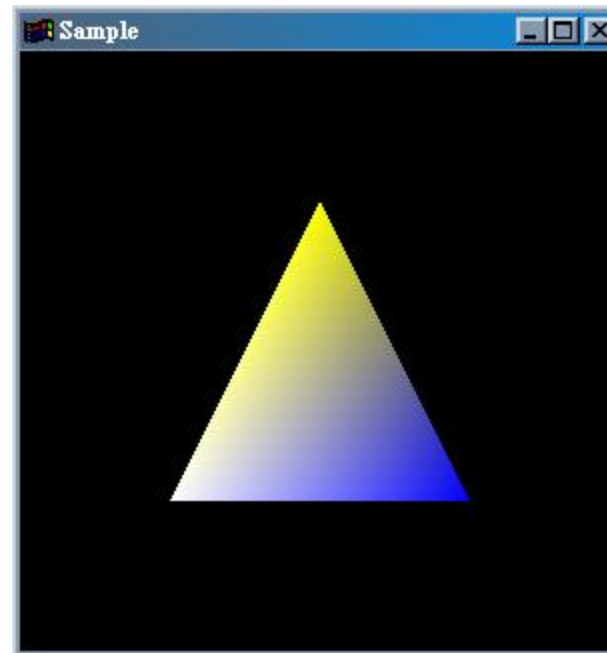
Color

- ▶ Clearing the Window
 - ▶ Use **glClearColor()** or **glClearIndex()** to specify the clear color.
 - ▶ Call **glClear(GL_COLOR_BUFFER_BIT);** if need to clear the color buffer.
- ▶ Specifying shading model
 - ▶ **glShadeModel(GLenum mode);**
 - ▶ The parameter is either GL_FLAT or GL_SMOOTH (default).

Color



GL_FLAT



GL_SMOOTH

Hidden-Surface Removal

- ▶ How to apply HSR in OpenGL
 - ▶ `glutInitDisplayMode(GLUT_DEPTH);`
 - ▶ `glEnable(GL_DEPTH_TEST);`
- ▶ In display function
 - ▶ `glClear(GL_DEPTH_BUFFER_BIT);`

Light in OpenGL

- ▶ **Ambient**
 - ▶ Light that is been scattered by the environment.
- ▶ **Diffuse**
 - ▶ Light that come from one direction.
- ▶ **Specular**
 - ▶ Light that come from particular direction that tend to bounce off the surface in a preferred direction.

Material in OpenGL

▶ Material

- ▶ Describe the percentages of the incoming red, green, blue light it reflects on a surface.

▶ **Diffuse**

- ▶ The most important role in determining what you perceive the color of an object to be.
- ▶ Affected by the color of the incident diffuse light and the angle of the incident light relative to the normal direction

▶ **Ambient**

- ▶ Affected by the global ambient light and ambient light from individual light sources.

OpenGL Light and Material

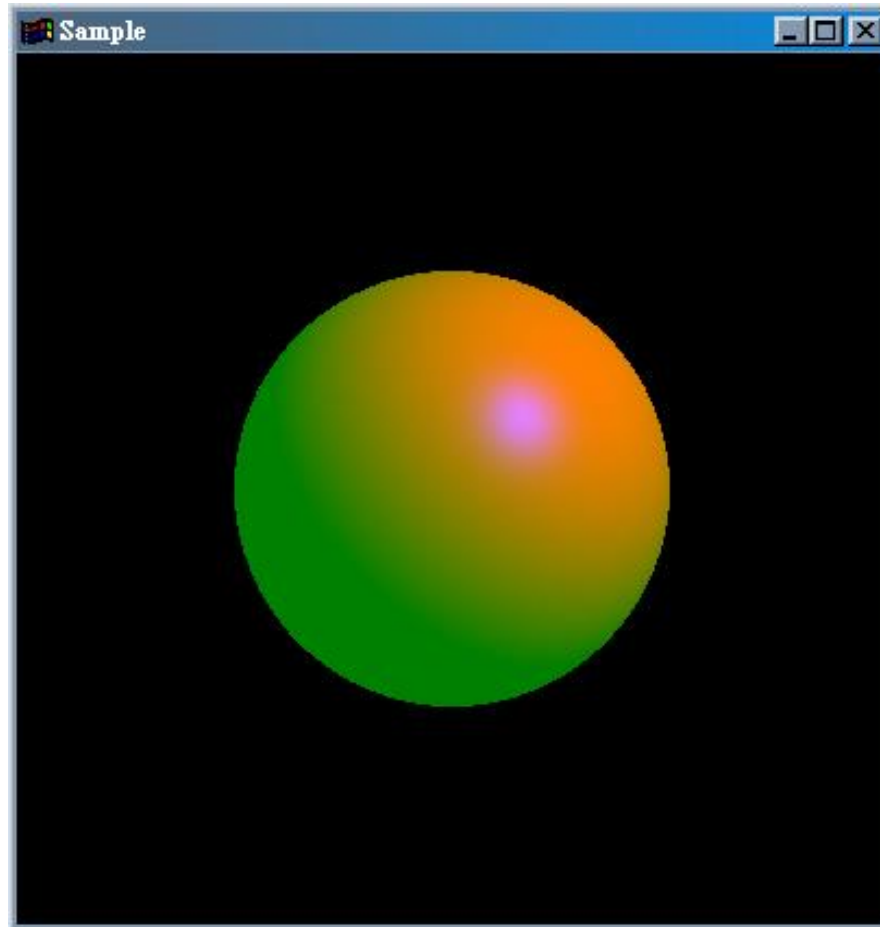
▶ **Specular & Shininess**

- ▶ Specular reflection from an object produces highlights.
- ▶ The amount of specular reflection seen by a viewer does depend on the location of the viewpoint.
- ▶ Shininess control the size and brightness of the highlight.

▶ **Emission**

- ▶ Make an object appear to be giving off light of that color.
- ▶ Since most real-world objects (except lights) don't emit light, you'll probably use this feature mostly to simulate lamps and other light sources in a scene.

OpenGL Light and Material



Ambient : Green

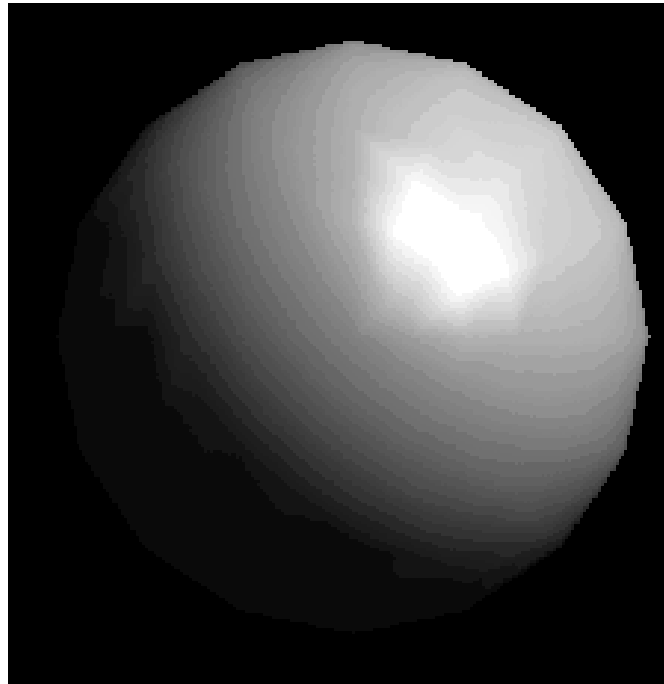
Diffuse : Red

Specular: Blue

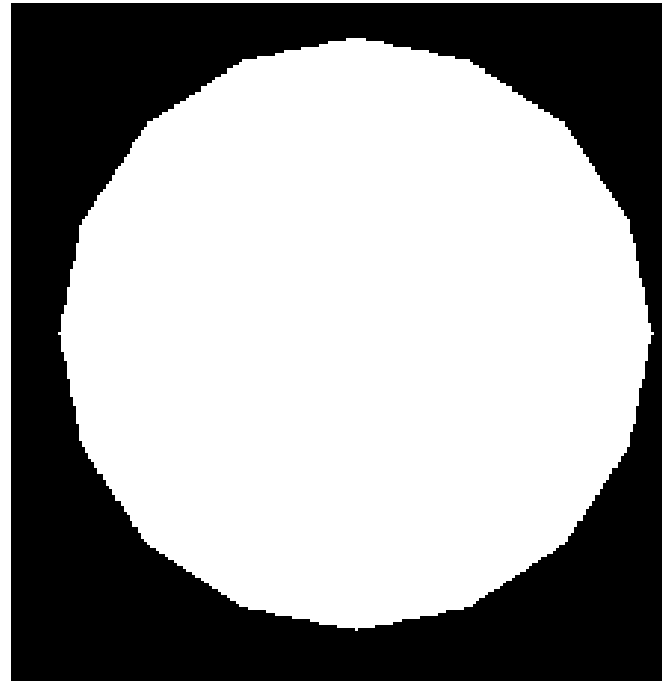
Put light into OpenGL

- ▶ How to apply lighting in OpenGL
 - ▶ Use **glEnable(GL_LIGHTING);** to enable lighting.
 - ▶ Enable and set properties for light sources.
 - ▶ Select a lighting model.
 - ▶ Define material properties for objects in the scene.
 - ▶ Assign normal vectors for each vertex of every object.

Lighting Sample



Lighting enabled



Lighting disabled

Lighting: Assign Normal Vector

- ▶ Assign normal vectors for each vertex of every object
 - ▶ **glNormal{34}{isfd}[v](TYPE* normal);**
 - ▶ Assign normal vector for vertices (the normal vector should be assigned before you assign vertices).
 - ▶ Normal vectors must be normalized. OpenGL can automatically normalize normal vector by **glEnable(GL_NORMALIZE);**
 - ▶ In the example, the normals for the sphere are defined as part of the **glutSolidSphere()** routine.

Sample: Assign Normal Vector

```
void GL_Display()
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glBegin(GL_POLYGON);
    glNormal3f(0.0, 1.0, 0.0);
    glVertex3f(1.0, 0.0, 0.0);
    glVertex3f(0.0, 0.0, -1.0);
    glVertex3f(-1.0, 0.0, 0.0);
    glEnd();
    glFlush();
}
```

Lighting: Light Source

- ▶ Enable and set properties for light sources
 - ▶ Use **glEnable(GL_LIGHT0);** to enable light zero. You can use at most 8 light sources in OpenGL spec. (GL_LIGHT0~GL_LIGHT7)
 - ▶ **glLight{if}[v](GLenum *light*, GLenum *pname*, TYPE *param*);**
 - ▶ Specify the attribute of light
 - ▶ *light* can be GL_LIGHT0 ~ GL_LIGHT7
 - ▶ *pname* is the characteristic of the light
 - ▶ *param* is the value of pname.

Lighting: Light Source

Pname	Def. Value	Meaning
GL_AMBIENT	(0.0, 0.0, 0.0, 0.0)	ambient RGBA intensity of light
GL_DIFFUSE	(1.0, 1.0, 1.0, 1.0)	diffuse RGBA intensity of light
GL_SPECULAR	(1.0, 1.0, 1.0, 1.0)	specular RGBA intensity of light
GL_POSITION	(0.0, 0.0, 1.0, 0.0)	(x, y, z, w) position of light
GL_SPOT_DIRECTION	(0.0, 0.0, -1.0)	(x, y, z) direction of spotlight
GL_SPOT_EXPONENT	0.0	spotlight exponent
GL_SPOT_CUTOFF	180.0	spotlight cutoff angle
GL_CONSTANT_ATTENUATION	1.0	constant attenuation factor
GL_LINEAR_ATTENUATION	0.0	linear attenuation factor
GL_QUADRATIC_ATTENUATION	0.0	quadratic attenuation factor

Lighting: Light Source

► Color

- The default values listed for GL_DIFFUSE and GL_SPECULAR apply only to GL_LIGHT0.
- For other lights, the default value is (0.0, 0.0, 0.0, 1.0) for both GL_DIFFUSE and GL_SPECULAR.

► Ex.

- `GLfloat light_ambient[4] = {0.0, 0.0, 1.0, 0.0};`
- `glLightfv(GL_LIGHT0, GL_AMBIENT, light_ambient);`

► Position (x, y, z, w)

- W is **ZERO**, **Directional light**, (x,y,z) specify its direction.
- W is **NONZERO**, **Positional light**, (x,y,z) specify the location of the light source.

Lighting: Light Source

▶ How to create spot light

- ▶ Define your light as positional light
- ▶ Define light spot direction

```
GLfloat spot_direction[] = { -1.0, -1.0, 0.0 };
```

```
glLightfv(GL_LIGHT0, GL_SPOT_DIRECTION, spot_direction);
```

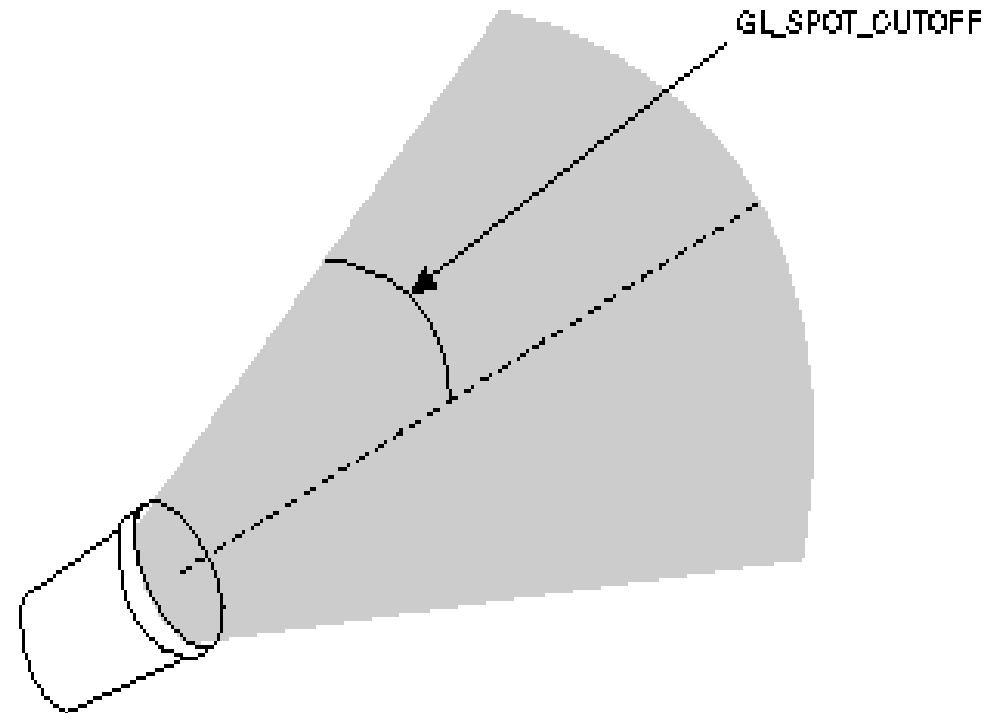
- ▶ Define light spot exponent

```
glLightf(GL_LIGHT1, GL_SPOT_EXPONENT, 2.0);
```

- ▶ Define light spot cutoff

The value for **GL_SPOT_CUTOFF** is restricted to being within the range [0.0,90.0] (unless it has the special value 180.0 (default)).

Lighting: Light Source



Lighting: Light Source

- ▶ Control a light's position and direction
 - ▶ OpenGL treats the position and direction of a light source just as it treats the position of a geometric primitive.
 - ▶ **MODELVIEW** Transformation will be applied.
- ▶ Three types of control
 - ▶ A light position that remains fixed
 - ▶ A light that moves around a stationary object
 - ▶ A light that moves along with the viewpoint

Lighting: Light Source

► Keeping the Light Stationary

```
glViewport (0, 0, (GLsizei) w, (GLsizei) h);
glMatrixMode (GL_PROJECTION);
glLoadIdentity();
if (w <= h)
    glOrtho (-1.5, 1.5, -1.5*h/w, 1.5*h/w, -10.0, 10.0);
else
    glOrtho (-1.5*w/h, 1.5*w/h, -1.5, 1.5, -10.0, 10.0);
glMatrixMode (GL_MODELVIEW);
glLoadIdentity();
...
/* later in init() */
GLfloat light_position[] = { 1.0, 1.0, 1.0, 1.0 };
glLightfv(GL_LIGHT0, GL_POSITION, position);
/* NO other MODELVIEW transformation is set...*/
```

Lighting: Light Source

► Independently Moving the Light

```
static GLdouble spin;
void display(void)
{
    GLfloat light_position[] = { 0.0, 0.0, 1.5, 1.0 };
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glPushMatrix();
    gluLookAt (0.0, 0.0, 5.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0);
    glPushMatrix();
    glRotated(spin, 1.0, 0.0, 0.0);
    glLightfv(GL_LIGHT0, GL_POSITION, light_position);
    glPopMatrix();
    glutSolidTorus (0.275, 0.85, 8, 15);
    glPopMatrix();
    glFlush();
}
```

Lighting: Light Source

- ▶ Moving the light source together with your viewpoint

```
GLfloat light_position() = { 0.0, 0.0, 0.0, 1.0 };
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
glLightfv(GL_LIGHT0, GL_POSITION, light_position);
.....
static GLdouble ex, ey, ez, upx, upy, upz;
void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glPushMatrix();
    gluLookAt (ex, ey, ez, 0.0, 0.0, 0.0, upx, upy, upz);
    glutSolidTorus (0.275, 0.85, 8, 15);
    glPopMatrix();
    glFlush();
}
```

Lighting: Lighting Model

- ▶ Selecting a lighting model

- ▶ **`void glLightModel{if}[v](GLenum pname, TYPE[*] param);`**

- ▶ Sets properties of the lighting model.

- ▶ The characteristic of the lighting model being set is defined by pname, which specifies a named parameter.

- ▶ param indicates the values to which the pname characteristic is set

Lighting: Lighting Model

Pname	Def. Value	Meaning
GL_LIGHT_MODEL_AMBIENT	(0.2, 0.2, 0.2, 1.0)	ambient RGBA intensity of the entire scene
GL_LIGHT_MODEL_LOCAL_VIEWER	0.0 or GL_FALSE	how specular reflection angles are computed
GL_LIGHT_MODEL_TWO_SIDE	0.0 or GL_FALSE	choose between one-sided or two-sided lighting

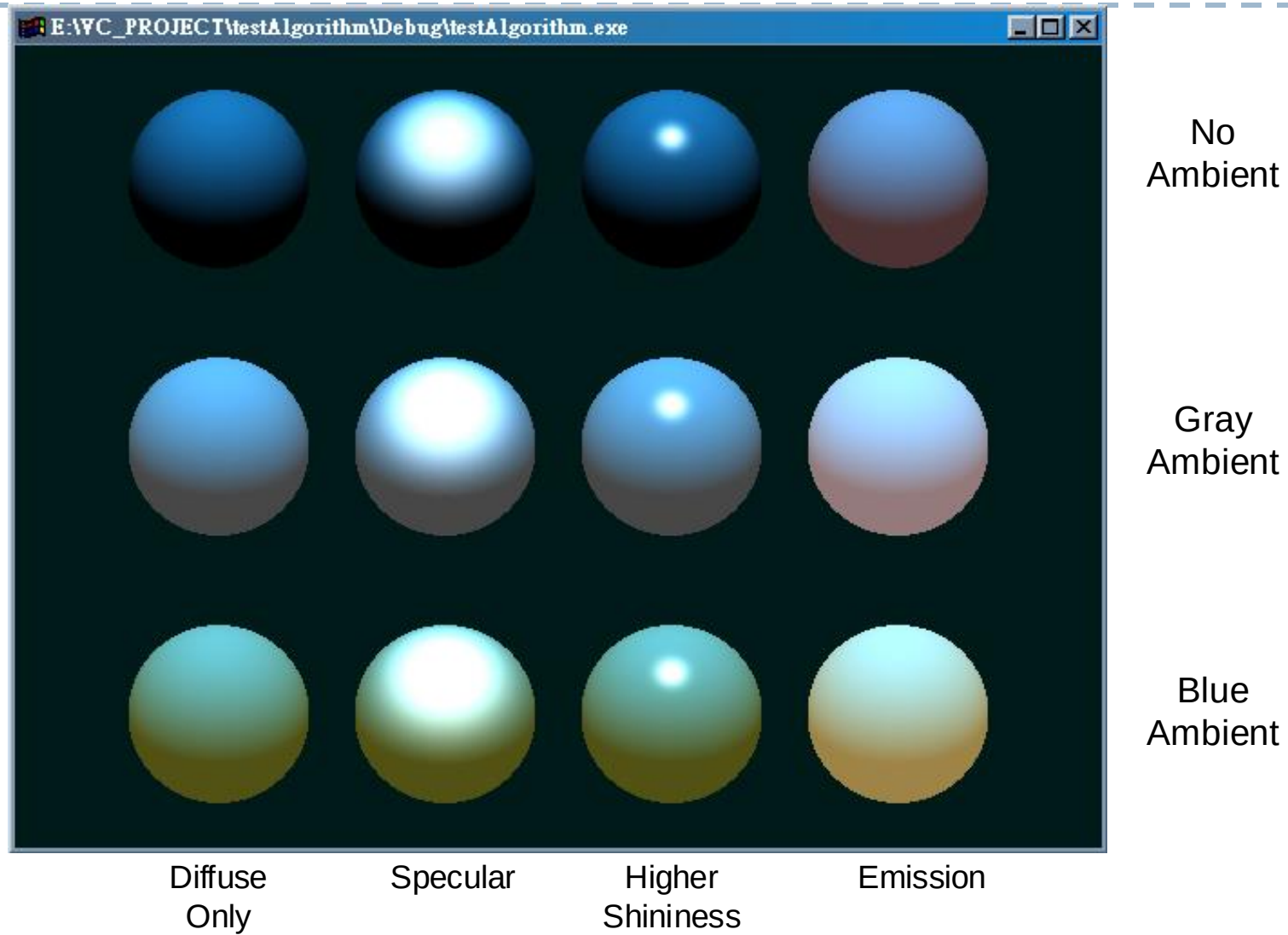
Lighting: Material

- ▶ Define material properties for objects in the scene.
 - ▶ **`void glMaterial{if}[v](GLenum face, GLenum pname, TYPE[*] param);`**
 - ▶ Specifies a current material property for use in lighting calculations.
 - ▶ face can be **`GL_FRONT`**, **`GL_BACK`**, or **`GL_FRONT_AND_BACK`**.
 - ▶ The particular material property being set is identified by pname and the desired values for that property are given by param.

Lighting: Material

Pname	Def. Value	Meaning
GL_AMBIENT	(0.2, 0.2, 0.2, 1.0)	ambient color of material
GL_DIFFUSE	(0.8, 0.8, 0.8, 1.0)	diffuse color of material
GL_AMBIENT_AND_DIFFUSE		ambient and diffuse color of material
GL_SPECULAR	(0.0, 0.0, 0.0, 1.0)	specular color of material
GL_SHININESS	0.0	specular exponent
GL_EMISSION	(0.0, 0.0, 0.0, 1.0)	emissive color of material

Lighting: Material



Lighting: Material

- ▶ Performance issue
 - ▶ **glMaterialfv();** is called repeatedly to set different material properties for different objects.
 - ▶ **glMaterial*()** has a performance cost associate with its use.
 - ▶ Another technique for minimizing performance costs associated with changing material properties is to use **glColorMaterial();**

Lighting: Material

- ▶ **glColorMaterial(GLenum *face*, GLenum *mode*);**
 - ▶ Causes the material property of the specified material face to track the value of the current color at all times.
 - ▶ face parameter can be GL_FRONT, GL_BACK, or GL_FRONT_AND_BACK (default).
 - ▶ mode parameter can be GL_AMBIENT, GL_DIFFUSE, GL_AMBIENT_AND_DIFFUSE (the default), GL_SPECULAR, or GL_EMISSION.
 - ▶ A change to the current color (using glColor*()) immediately updates the specified material properties.
 - ▶ Use **glEnable(GL_COLOR_MATERIAL);** to enable color material.
 - ▶ **glColorMaterial()** has no effect on color-index lighting.

Ερωτήσεις

- ▶ <https://www.e-ce.uth.gr/studies/undergraduate/courses/ece433/>
- ▶ Ιστοσελίδα μαθήματος :
https://eclass.uth.gr/courses/E-CE_U_244/
- ▶ Π. Τσομπανοπούλου, 416, yota@uth.gr